

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

UNIT 5
KẾT NỐI INTERNET

ONE LOVE. ONE FUTURE.

Mục đích

- Hiểu và sử dụng được các kỹ thuật lập trình bất đồng bộ sử dụng thư viện coroutine
- Sử dụng được các hàm runBlocking(), launch(), async()
- Có thể gọi được REST API bằng thư viện Retrofit và phân tích cú pháp JSON
- Tải ảnh từ internet và hiển thị bằng thư viện Coil
- Hiểu và thiết kế được lớp dữ liệu trong kiến trúc ứng dụng Android

Nội dung

- Lập trình bất đồng bộ trong Kotlin và Android sử dụng coroutine
- Kết nối Web API sử dụng thư viện Retrofit
- Lớp repository trong tầng dữ liệu
- Hiển thị ảnh sử dụng thư viện Coil

HUST

hust.edu.vn fb.com/dhbkhn

PHẦN 1

KẾT NỐI INTERNET

- Giới thiệu về Coroutines trong Kotlin và Android
- Giới thiệu về HTTP/REST
- Lấy dữ liệu từ internet

Đồng bộ vs bất đồng bộ trong ứng dụng di động

ĐỒNG BỘ
Tác vụ chặn Main thread



UI tạm thời không phản hồi



Lệnh tiếp theo chờ tác vụ trước hoàn tất

BẤT ĐỒNG BỘ
Chờ kết quả, UI vẫn phản hồi



Vẫn cuộn và tương tác



Coroutine có thể tạm ngưng mà không chặn Main thread

Kỹ thuật lập trình bất đồng bộ trong Android

- **Thread + Handler/Looper:** phù hợp tác vụ cấp thấp; cần tự quản lý đồng bộ dữ liệu và vòng đời.
- **ExecutorService / RxJava:** dùng cho chạy song song hoặc luồng dữ liệu phản ứng, thường gặp trong dự án legacy.
- **Kotlin Coroutines + Flow:** cách tiếp cận hiện đại với suspend, Dispatcher, ViewModelScope và lifecycleScope.
- **WorkManager:** cho công việc nền cần đảm bảo chạy, có ràng buộc mạng/sạc và tồn tại sau khi app đóng.
- **Nguyên tắc triển khai:** hủy tác vụ đúng vòng đời, xử lý lỗi/timeout, chuyển về Main thread trước khi cập nhật UI.

Kotlin Coroutines

- **Khái niệm:** Coroutines là cơ chế chạy tác vụ bất đồng bộ nhẹ hơn Thread, giúp viết code tuần tự nhưng không chặn UI.
- Cách sử dụng đơn giản và hiệu quả hơn so với các kỹ thuật truyền thống như Thread, AsyncTask, Callback.
- Sử dụng ít bộ nhớ hơn
- Cho phép chuyển đổi linh hoạt giữa các ngữ cảnh (chạy nền, cập nhật giao diện)
- Được sử dụng để thực hiện kết nối CSDL (ROOM), kết nối Internet.

Ví dụ đầu tiên

Để tạo và chạy coroutine cần thêm các thư viện vào project.

Đối với project Kotlin

```
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core-jvm:1.5.0")
```

Đối với project Android

```
implementation 'org.jetbrains.kotlinx:kotlinx-coroutines-core:1.2.1'

implementation 'org.jetbrains.kotlinx:kotlinx-coroutines-android:1.1.1'
```

Ví dụ đầu tiên

Tạo file **test.kt** và chạy đoạn mã sau:

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    launch {
        delay(5000L)
        println("World")
    }

    println("Hello")
}
```

Kết quả thực hiện:

```
Hello
World
```

=> Từ “World” xuất hiện sau từ “Hello” 5 giây.

Ví dụ đầu tiên

Tạo file **test.kt** và chạy đoạn mã sau:

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    launch {
        delay(5000L)
        println("World")
    }

    println("Hello")
}
```

Giải thích:

- **launch** – tạo và chạy một coroutine mới đồng thời với phần còn lại của mã nguồn
- **delay** – hàm suspend dừng coroutine trong một khoảng thời gian, việc tạm dừng coroutine không làm dừng luồng, cho phép các coroutine khác vẫn có thể hoạt động
- **runBlocking** – được dùng để kết nối mã nguồn không dùng coroutine và mã nguồn sử dụng coroutine, tạo và chạy một coroutine, hàm trả về kết quả khi các coroutine bên trong kết thúc

Ví dụ đầu tiên

Cập nhật file **test.kt** như sau:

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    launch { doWorld() }
    println("Hello")
}

suspend fun doWorld() {
    delay(5000L)
    println("World")
}
```

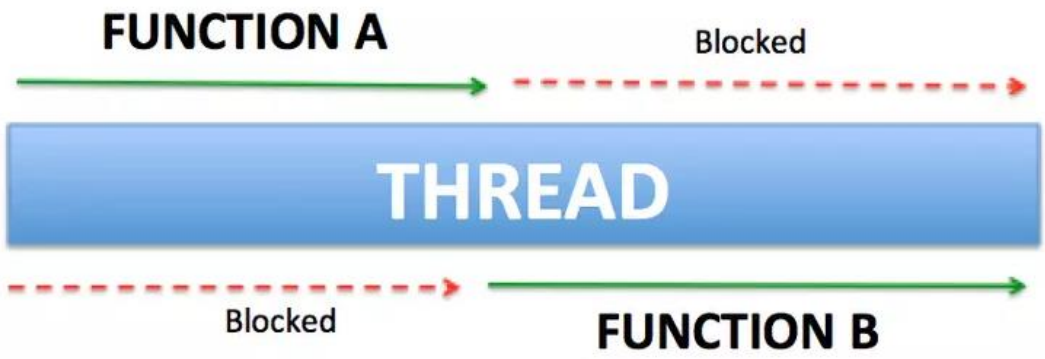
doWorld() là ví dụ về hàm **suspend** được gọi trong các coroutines, nội dung hàm có thể gọi các hàm suspend khác cho phép tạm dừng coroutine.

Khái niệm hàm suspend

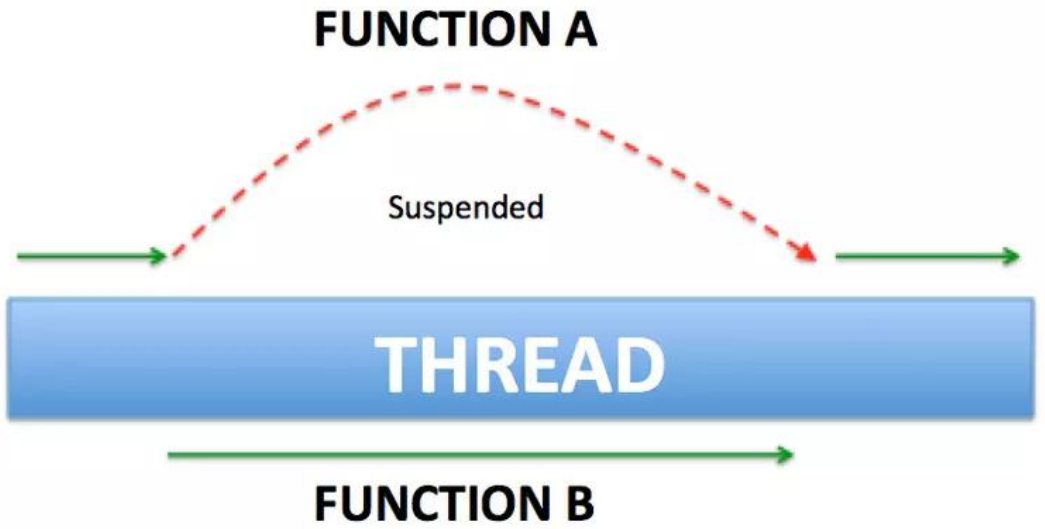
- Là hàm được sử dụng trong các coroutines hoặc trong các hàm suspend khác.
- Có khả năng tạm dừng việc thực thi và tiếp tục khi cần thiết.

Khái niệm hàm suspend

Hàm block: dừng luồn hàm khác phải chờ.



Hàm suspend: nếu chưa có kết quả thì sẽ tạm dừng nhưng không dừng luồng, cho phép các hàm khác không phải chờ.



Cách thức khởi tạo coroutine

Để tạo và chạy các coroutine, có thể sử dụng các hàm

- **launch()** – khởi tạo coroutine thực hiện công việc mà không trả về kết quả
- **runBlocking()** – khởi tạo coroutine và đợi cho đến khi hoàn tất
- **async()** – khởi tạo coroutine thực hiện công việc và có trả về kết quả

Hàm launch()

```
fun CoroutineScope.launch(
    context: CoroutineContext = EmptyCoroutineContext,
    start: CoroutineStart = CoroutineStart.DEFAULT,
    block: suspend CoroutineScope.() -> Unit
): Job
```

- Tạo và chạy coroutine mới mà không block luồng hiện tại và trả về kết quả là đối tượng tham chiếu đến coroutine thuộc lớp **Job**. Quản lý coroutine thông qua đối tượng này.

- Ngữ cảnh thực thi được kế thừa từ **CoroutineScope**, có thể được bổ sung bằng tham số **context**.

- Mặc định coroutine được xếp lịch để thực hiện ngay lập tức, hoặc theo các lựa chọn khác bằng tham số **start**.

Ví dụ

```
launch {
    delay(2000L)
    println("Third message!")
}
launch {
    delay(1000L)
    println("Second message!")
}
println("First message!")
```

Kết quả khi chạy:

- “First message!” được hiển thị đầu tiên.
- “Second message!” hiển thị sau 1 giây.
- “Third message!” hiển thị sau 2 giây.

Ví dụ

```
val job1 = lifecycleScope.launch(Dispatchers.IO) {
    // Thực hiện kết nối Internet
}

val job2 = lifecycleScope.launch(Dispatchers.Main) {
    // Cập nhật giao diện
}
```

- **job1** chạy ở luồng background, thực hiện kết nối CSDL, kết nối Internet
- **job2** chạy ở luồng giao diện, thực hiện cập nhật giao diện
- **lifecycleScope** là phạm vi gắn với activity, fragment

Hàm runBlocking()

```
expect fun <T> runBlocking(
    context: CoroutineContext = EmptyCoroutineContext,
    block: suspend CoroutineScope.() -> T
): T
```

- Tạo và chạy coroutine mới và dừng luồng cho đến khi hoàn thành.

- Hàm được thiết kế để chuyển đổi giữa kiểu mã nguồn blocking và mã nguồn suspending.

- Được sử dụng chủ yếu trong hàm **main()** với mục đích thử nghiệm.

Ví dụ

```
fun main() {
    runBlocking {
        delay(2000L)
        println("First message!")
    }
    println("Second message!")
}
```

Kết quả khi chạy:

- “First message!” hiển thị sau 2 giây.
- “Second message!” hiển thị sau cùng.

Chờ và hủy coroutine đang chạy

Hàm **join()** – chờ coroutine hoàn thành

Hàm **cancel()** – hủy thực hiện coroutine

Hàm **withTimeout()** – chạy coroutine trong khoảng thời gian giới hạn

Chờ coroutine hoàn thành

Sử dụng hàm **join()** của lớp **Job**

Ví dụ: **job1.join()** sẽ tạm dừng **job2** cho đến khi **job1** hoàn thành

```
runBlocking {
    val job1 = launch {
        repeat(10) {
            delay(1000L)
            println("Job1: ${it + 1}")
        }
    }
    val job2 = launch {
        println("Waiting for job1 complete...")
        job1.join()
        repeat(10) {
            delay(1000L)
            println("Job2: ${it + 1}")
        }
    }
}
```

Hủy coroutine đang thực hiện

Sử dụng hàm **cancel()** của lớp **Job, CoroutineScope** hoặc **CoroutineContext**

```
fun CoroutineScope.cancel(cause: CancellationException? = null)
fun CoroutineScope.cancel(message: String, cause: Throwable? = null)
fun CoroutineContext.cancel(cause: CancellationException? = null)
fun Job.cancel(message: String, cause: Throwable? = null)
```

- Hủy các coroutine trong một phạm vi, ngữ cảnh hoặc một coroutine cụ thể. Các tham số kèm theo cho biết nguyên nhân việc hủy coroutine.
- Chú ý: hàm **cancel()** không hủy được coroutine chạy với ngữ cảnh **NonCancellable**

Ví dụ

```
runBlocking {
    val job1 = launch {
        repeat(10) {
            delay(1000L)
            println("Job1: ${it + 1}")
        }
    }

    val job2 = launch {
        println("Waiting for 5 seconds then cancel Job1")
        delay(5000L)
        job1.cancel()
    }
}
```

job2 hủy **job1** sau 5 giây, **job1** đếm đến 4 thì bị dừng.

Hàm withTimeout()

```
suspend fun <T> withTimeout(timeMillis: Long, block: suspend
CoroutineScope.() -> T): T
```

- Chạy đoạn chương trình (có thể bị suspend) trong một coroutine với thời gian giới hạn (tính theo ms).
- Nếu hết thời gian mà đoạn chương trình chưa hoàn thành thì tạo exception thuộc kiểu TimeoutCancellationException.

Ví dụ

```
runBlocking {
    withTimeout(5000L) {
        repeat(10) {
            delay(800L)
            println("Counting: ${it + 1}")
        }
    }
}
```

Kết quả thực hiện

```
Counting: 1
Counting: 2
Counting: 3
Counting: 4
Counting: 5
Counting: 6
Exception in thread "main" kotlinx.coroutines.TimeoutCancellationException: Timed out waiting for 5000 ms
```

Hàm async()

```
fun <T> CoroutineScope.async(
    context: CoroutineContext = EmptyCoroutineContext,
    start: CoroutineStart = CoroutineStart.DEFAULT,
    block: suspend CoroutineScope.() -> T
): Deferred<T>
```

- Được sử dụng để tạo và chạy một coroutine mới với các tham số tương tự hàm **launch()**
- Trả về kết quả trong tương lai của coroutine trong một đối tượng thuộc lớp **Deferred** (là lớp con của lớp Job)
- Kết quả của coroutine có được khi hoàn thành và được trả về bởi hàm **await()**

Hàm await()

Đợi cho đến khi có kết quả của đối tượng **Deferred** (coroutine được tạo bởi async hoàn thành)

```
runBlocking {
    val value1: Deferred<Int> = async {
        println("Compute value 1")
        delay(3000L)
        println("Return value 1")
        return@async Random.nextInt(10, 100)
    }

    val value2: Deferred<Int> = async {
        println("Compute value 2")
        delay(1000L)
        println("Return value 2")
        return@async Random.nextInt(10, 100)
    }

    println("Value 1: ${value1.await()}")
    println("Value 2: ${value2.await()}")
}
```

Kết quả khi chạy:

```
Compute value 1
Compute value 2
Return value 2
Return value 1
Value 1: 57
Value 2: 84
```

CoroutineContext

- Các coroutine luôn hoạt động trong ngữ cảnh được mô tả bởi giá trị thuộc kiểu CoroutineContext.
- Các thành phần chính của một CoroutineContext
  - Job – nắm giữ thông tin về lifecycle của coroutine
  - Dispatcher – quyết định luồng (thread) nào mà coroutine hoạt động trên đó
- Các loại dispatcher
  - Dispatchers.Main – chạy trên luồng giao diện
  - Dispatchers.IO – chạy trên luồng background
  - Dispatchers.Defaults – chạy trên luồng background
  - newSingleThreadContext("thread_name") – chạy trên luồng mới với tên là **thread_name**

Ví dụ

```
runBlocking {
    launch { // context of the parent, main runBlocking coroutine
        println("main runBlocking       : I'm working in thread ${Thread.currentThread().name}")
    }
    launch(Dispatchers.Default) { // will get dispatched to DefaultDispatcher
        println("Default                : I'm working in thread ${Thread.currentThread().name}")
    }
    launch(Dispatchers.IO) { // will get dispatched to DefaultDispatcher
        println("IO                     : I'm working in thread ${Thread.currentThread().name}")
    }
    launch(newSingleThreadContext("MyOwnThread")) { // will get its own new thread
        println("newSingleThreadContext : I'm working in thread ${Thread.currentThread().name}")
    }
}
```

Kết quả thực hiện:

```
Default                : I'm working in thread DefaultDispatcher-worker-1
IO                     : I'm working in thread DefaultDispatcher-worker-2
newSingleThreadContext : I'm working in thread MyOwnThread
main runBlocking       : I'm working in thread main
```

Chuyển đổi giữa các ngữ cảnh

Sử dụng hàm **withContext()**

```
runBlocking {
    println("Started in ${Thread.currentThread().name}")
    withContext(Dispatchers.IO) {
        println("Changed to ${Thread.currentThread().name}")
    }
    println("Back to ${Thread.currentThread().name}")
}
```

Kết quả thực hiện:

```
Started in main
Changed to DefaultDispatcher-worker-1
Back to main
```

CoroutineScope

- Là phạm vi mà các coroutine hoạt động, gắn liền với lifecycle của activity hoặc viewmodel
- Khi lifecycle của các đối tượng này kết thúc thì các coroutines đang chạy trong phạm vi tương ứng cũng bị kết thúc.
- Các kiểu phạm vi:
  - **CoroutineScope** – interface cơ bản, tạo scope với context tùy chọn
  - **GlobalScope** – được sử dụng để khởi tạo các coroutines bậc cao hoạt động suốt vòng đời của ứng dụng
  - **MainScope** – tạo scope với context là Main thread
  - **lifecycleScope** – scope gắn với Activity
  - **viewModelScope** – scope gắn với ViewModel

Ví dụ

```
val scope = CoroutineScope(Dispatchers.Default)
runBlocking(Dispatchers.Default) {
    scope.launch {
        repeat(10) {
            delay(1000L)
            println("Job1: ${it + 1}")
        }
    }
    scope.launch {
        repeat(10) {
            delay(800L)
            println("Job2: ${it + 1}")
        }
    }

    delay(5000L)
    scope.cancel()
}
```

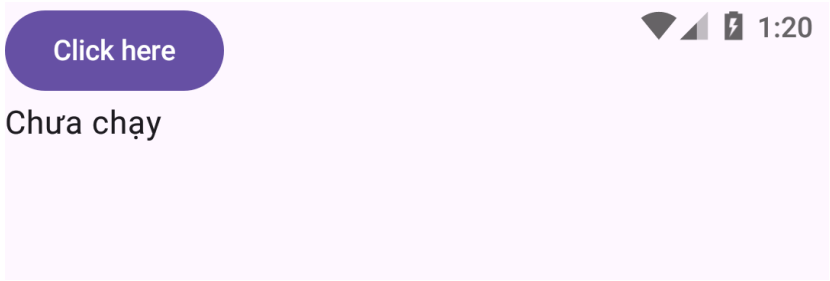
Cách sử dụng coroutines trong Jetpack Compose

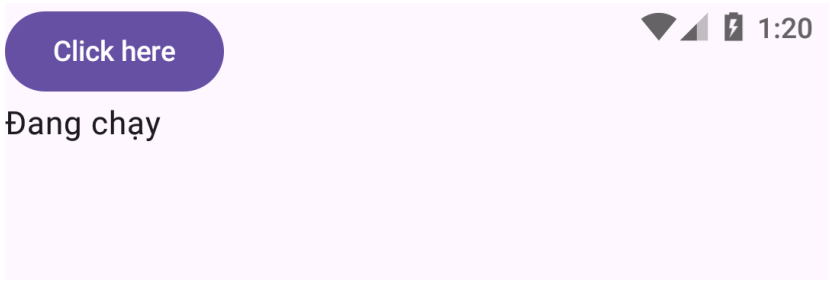
Trong Jetpack Compose, coroutines có thể được sử dụng trong 2 trường hợp (vẫn thuộc phạm vi activity):

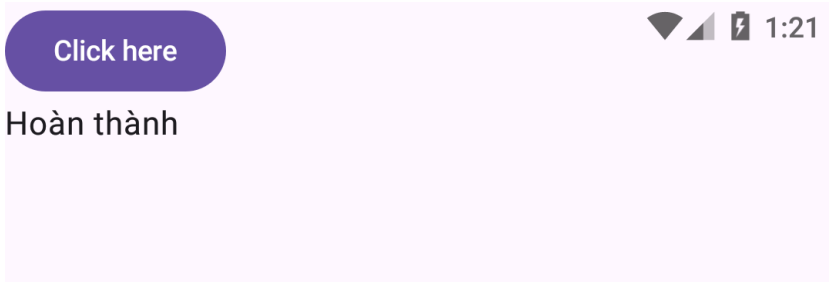
- Khi một sự kiện xảy ra: nút được bấm, danh sách được cuộn, …
- Khi một Composable xuất hiện hoặc một state thay đổi: tự chạy khi mở màn hình, …

Chạy coroutines khi bấm nút

```
val scope = rememberCoroutineScope()
var isProcessing by remember { mutableStateOf(false) }
var message by remember { mutableStateOf("Chưa chạy") }
Column() {
    Button(onClick = {
        scope.launch {
            isProcessing = true
            message = "Đang chạy"
            try {
                delay(5000.milliseconds)
                message = "Hoàn thành"
            } finally {
                isProcessing = false
            }
        }
    }) { Text("Click here") }
    Text(text = message)
}
```

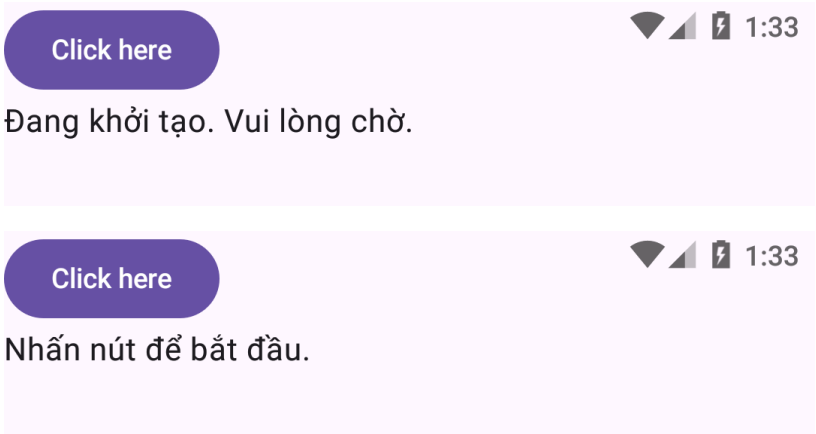






Tự chạy khi recomposition

```
LaunchedEffect(Unit) {
    message = "Đang khởi tạo. Vui lòng chờ."
    try {
        delay(5000.milliseconds)
        isProcessing = true
    } finally {
        message = "Nhấn nút để bắt đầu."
        isProcessing = false
    }
}
```



Tự chạy khi key thay đổi

```
var link by remember { mutableStateOf("") }
LaunchedEffect(link) {
    message = "Downloading"
    isProcessing = true

    try {
        // Download file
        message = "Done"
    } catch (ex: Exception) {
        message = "Failed"
    } finally {
        isProcessing = false
    }
}
```

Codelab

https://developer.android.com/codelabs/basic-android-kotlin-compose-coroutines-android-studio

HUST

hust.edu.vn fb.com/dhbkhn

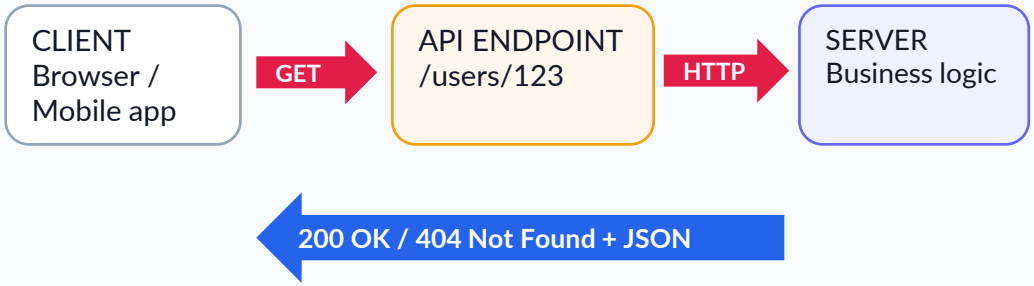
Lấy dữ liệu từ internet

HTTP & REST: nền tảng giao tiếp của Web API

HTTP là "ngôn ngữ vận chuyển"; REST là kiểu thiết kế giúp API dễ hiểu, dễ mở rộng và tách biệt client–server.

1. Luồng HTTP request / response

Client gửi yêu cầu qua HTTP; server xử lý tài nguyên rồi trả phản hồi có mã trạng thái và dữ liệu.



HTTP message

Headers	Body	Status code
Content-Type,	JSON / form	2xx • 4xx • 5xx

2. REST biến HTTP thành API có cấu trúc

GET	đọc tài nguyên
POST	tạo mới
PUT/PATCH	cập nhật
DELETE	xóa

Nguyên tắc cốt lõi

- Tài nguyên có URL rõ ràng
- Stateless: mỗi request tự đủ ngữ cảnh
- Dùng chuẩn HTTP thay vì lệnh tùy biến
- Phản hồi thường ở JSON

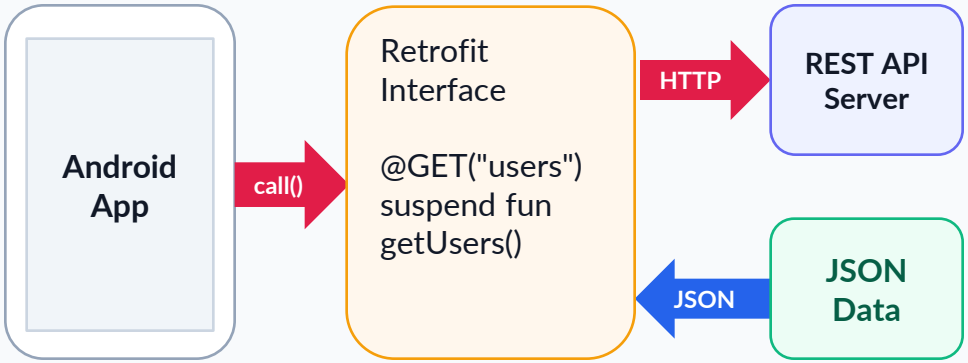
Ghi nhớ: HTTP trả lời "gửi như thế nào", REST trả lời "thiết kế API ra sao".

Giới thiệu thư viện Retrofit

Retrofit biến HTTP API thành interface Kotlin/Java rõ ràng, tự parse JSON và tích hợp tốt với OkHttp, Gson/Moshi, Coroutine hoặc RxJava.

Cách Retrofit nằm trong luồng Android → API

Ứng dụng chỉ gọi hàm interface; Retrofit tạo request, gửi qua OkHttp và ánh xạ response thành object.



OkHttp xử lý kết nối, cache, timeout, interceptor

Vì sao dùng Retrofit?

1. **Ít mã dư thừa**
   Định nghĩa endpoint bằng annotation: @GET, @POST, @Body...
2. **Kiểu dữ liệu an toàn**
   Response được map về data class/model rõ kiểu.
3. **Dễ mở rộng**
   Tách API service, repository và UI theo kiến trúc sạch.
4. **Dễ kiểm thử**
   Có thể mock interface hoặc thay baseUrl khi test.

Các bước khai báo và sử dụng thư viện Retrofit

1 Khai báo quyền Internet

AndroidManifest.xml

```
<uses-permission
android:name="android.permission.INTERNET" />
```

Đặt trong <manifest>, ngoài <application>.

2 Định nghĩa model dữ liệu

```
data class Product(
  val id: Int,
  val name: String
)
```

Ánh xạ dữ liệu JSON

3 Khai báo interface API

```
interface ApiService {
  @GET("products")
  suspend fun getProducts():
    List<Product>
}
```

4 Tạo đối tượng Retrofit

```
val retrofit = Retrofit.Builder()
    .baseUrl(BASE_URL)
    .addConverterFactory(
      GsonConverterFactory.create()
    )
    .build()
```

URL gốc + bộ chuyển đổi JSON

5 Tạo API service

```
val api = retrofit.create(
   ApiService::class.java
)
```

Retrofit triển khai interface

6 Gọi API bằng Coroutines

```
viewModelScope.launch {
    try {
        val products = api.getProducts()
        uiState = Success(products)
    } catch (e: IOException) {
        uiState = Error
    }
}
```

Hàm suspend chờ kết quả

Luồng dữ liệu



Khai báo quyền trong AndroidManifest.xml

- Bảo vệ sự riêng tư của người sử dụng thiết bị.
- Khai báo bằng thẻ <uses-permission> trong file AndroidManifest.xml
- Các quyền sử dụng cho kết nối Internet

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission
android:name="android.permission.ACCESS_NETWORK_STATE" />
```

Khai báo thư viện trong Gradle

Trong file **build.gradle.kts** mức project

```
plugins {
    id("com.google.devtools.ksp") version "2.3.4" apply false
}
```

Trong file **build.gradle.kts** mức module

```
plugins {
    id("com.google.devtools.ksp")
}

dependencies {
    implementation("com.squareup.retrofit2:retrofit:3.0.0")
    implementation("com.squareup.retrofit2:converter-moshi:3.0.0")

    implementation("com.squareup.moshi:moshi:1.15.2")
    implementation("com.squareup.moshi:moshi-kotlin:1.15.2")
    ksp("com.squareup.moshi:moshi-kotlin-codegen:1.15.2")
}
```

Web API thử nghiệm

JSONPlaceholder cung cấp REST API với dữ liệu giả định.

Method	Endpoint
GET	/posts
GET	/posts/1
GET	/posts/1/comments
GET	/comments?postId=1
POST	/posts
PUT	/posts/1
PATCH	/posts/1
DELETE	/posts/1


```
{
    "userId": 1,
    "id": 1,
    "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
    "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit
molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
}
```

Tạo model dữ liệu

```
@JsonClass(generateAdapter = true)
data class Post(
    val id: Int,
    val userId: Int,
    val title: String,
    val body: String
)
```

Định nghĩa các hàm dịch vụ API

```
interface PostService {
    @GET("posts")
    suspend fun getPosts(): List<Post>

    @GET("posts/{postId}")
    suspend fun getPost(@Path("postId") postId: Int): Post

    @POST("posts")
    suspend fun createPost(@Body post: Post): Int

    @PUT("posts/{postId}")
    suspend fun updatePost(@Path("postId") postId: Int, @Body post: Post): Int

    @DELETE("posts/{postId")
    suspend fun deletePost(@Path("postId") postId: Int): Int
}
```

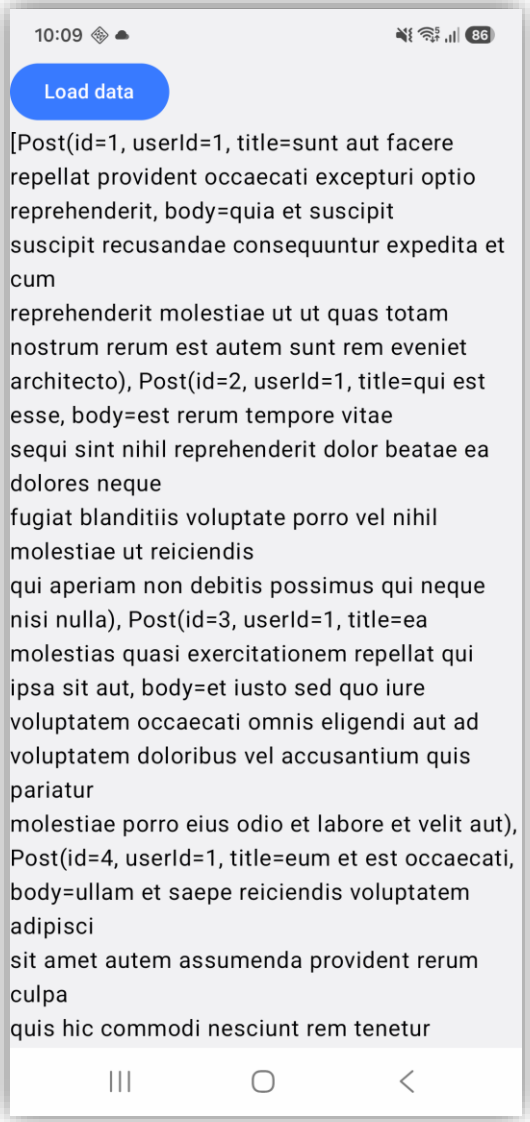
Tạo đối tượng API service

```
object PostAPI {
    private val moshi = Moshi.Builder()
        .add(KotlinJsonAdapterFactory())
        .build()
    private val retrofit = Retrofit.Builder()
        .addConverterFactory(MoshiConverterFactory.create(moshi))
        .baseUrl("https://jsonplaceholder.typicode.com/")
        .build()

    val service: PostService by lazy {
        retrofit.create(PostService::class.java)
    }
}
```

Sử dụng trong activity

```
@Composable
fun Test1(modifier: Modifier = Modifier) {
    val scope = rememberCoroutineScope()
    var data by remember { mutableStateOf("") }
    Column(modifier = modifier) {
        Button(onClick = {
            scope.launch {
                val posts = PostAPI.service.getPosts()
                data = posts.toString()
            }
        }) {
            Text("Load data")
        }
        Text(
            text = data
        )
    }
}
```



Sử dụng trong View Model

Khai báo UiState

```
data class PostUiState (
    val posts: List<Post> = emptyList(),
    val isLoading: Boolean = false,
    val error: String? = null
)
```

Sử dụng trong View Model (tiếp)

```
class PostViewModel: ViewModel() {
    private val _uiState = MutableStateFlow(PostUiState())
    val uiState: StateFlow<PostUiState> = _uiState.asStateFlow()

    fun loadPosts() {
        if (_uiState.value.isLoading) return
        _uiState.update {
            it.copy(isLoading = true, error = null)
        }
        viewModelScope.launch {
            try {
                val posts = PostAPI.service.getPosts()
                _uiState.update {
                    it.copy(posts = posts, isLoading = false, error = null)
                }
            } catch (e: Exception) {
                _uiState.update {
                    it.copy(isLoading = false, error = e.message)
                }
            }
        }
    }
}
```

Khai báo hàm tải dữ liệu trong ViewModel
+ Đánh dấu trạng thái bắt đầu, kết thúc việc tải dữ liệu.
+ Bắt các lỗi có thể xảy ra.

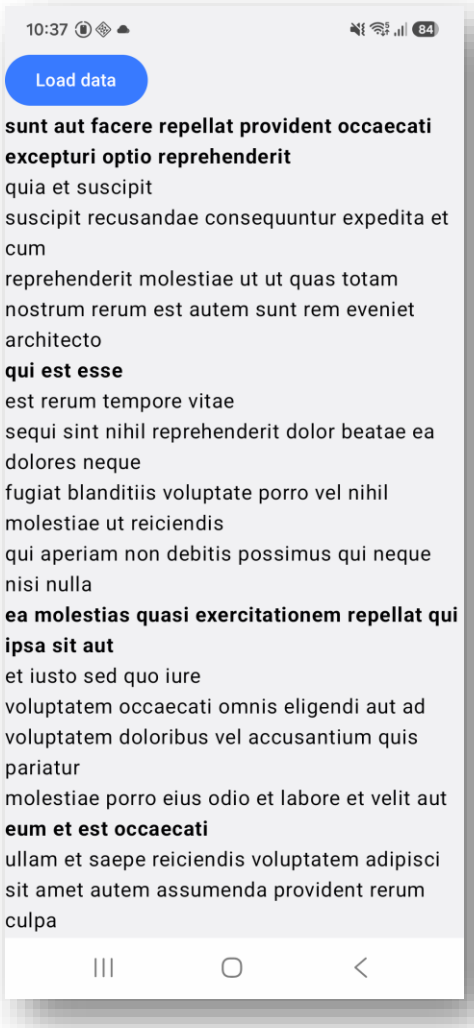
Sử dụng trong View Model (tiếp)

```
@Composable
fun Test2(postViewModel: PostViewModel = viewModel(), modifier: Modifier = Modifier) {
    val uiState by postViewModel.uiState.collectAsState()
    Column(modifier = modifier) {
        Button(onClick = {  postViewModel.loadPosts() }) {
            Text("Load data")
        }

        if (uiState.isLoading) CircularProgressIndicator()

        uiState.error?.let { message -> Text(text = message) }


        LazyColumn {
            items(items = uiState.posts) { post ->
                Column {
                    Text(text = post.title, fontWeight = FontWeight.Bold)
                    Text(text = post.body)
                }
            }
        }
    }
}
```



Codelab

https://developer.android.com/codelabs/basic-android-kotlin-compose-getting-data-internet

HUST

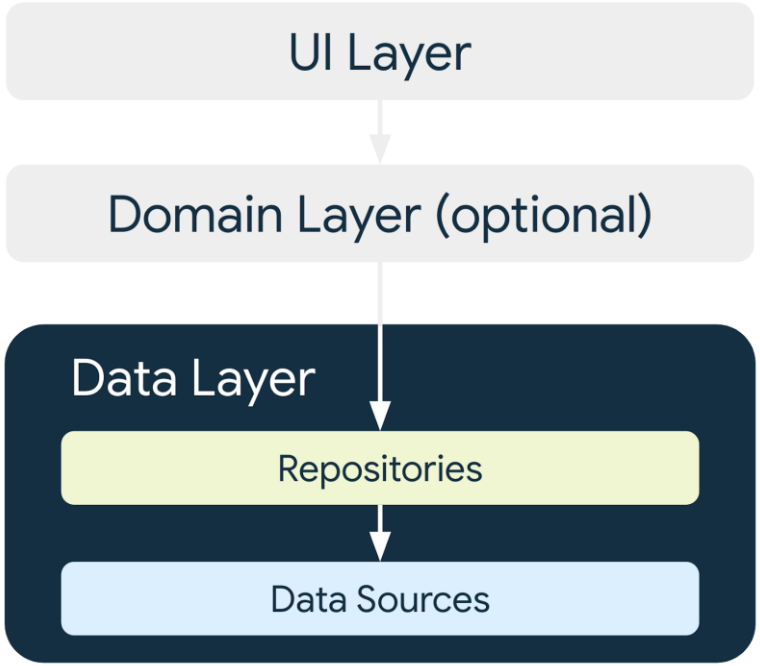
hust.edu.vn fb.com/dhbkhn

PHẦN 2

- **Tìm hiểu về lớp Data trong kiến trúc ứng dụng.**
- Hiển thị ảnh từ internet

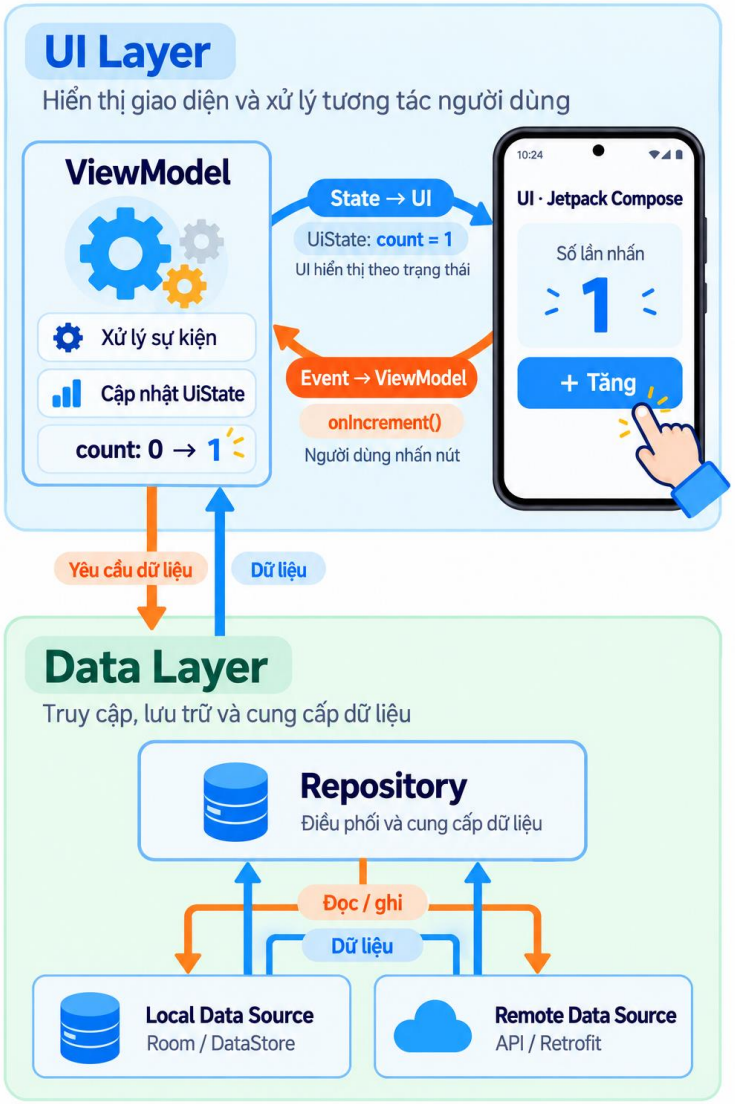
Data layer in app architecture

- Chịu trách nhiệm cung cấp dữ liệu cho UI / domain layer.
- Repositories đóng vai trò API thống nhất, che giấu chi tiết nguồn dữ liệu.
- Data sources có thể là REST API, Room database, DataStore, file hoặc cache.
- Mục tiêu: một "single source of truth" ổn định cho toàn bộ ứng dụng.



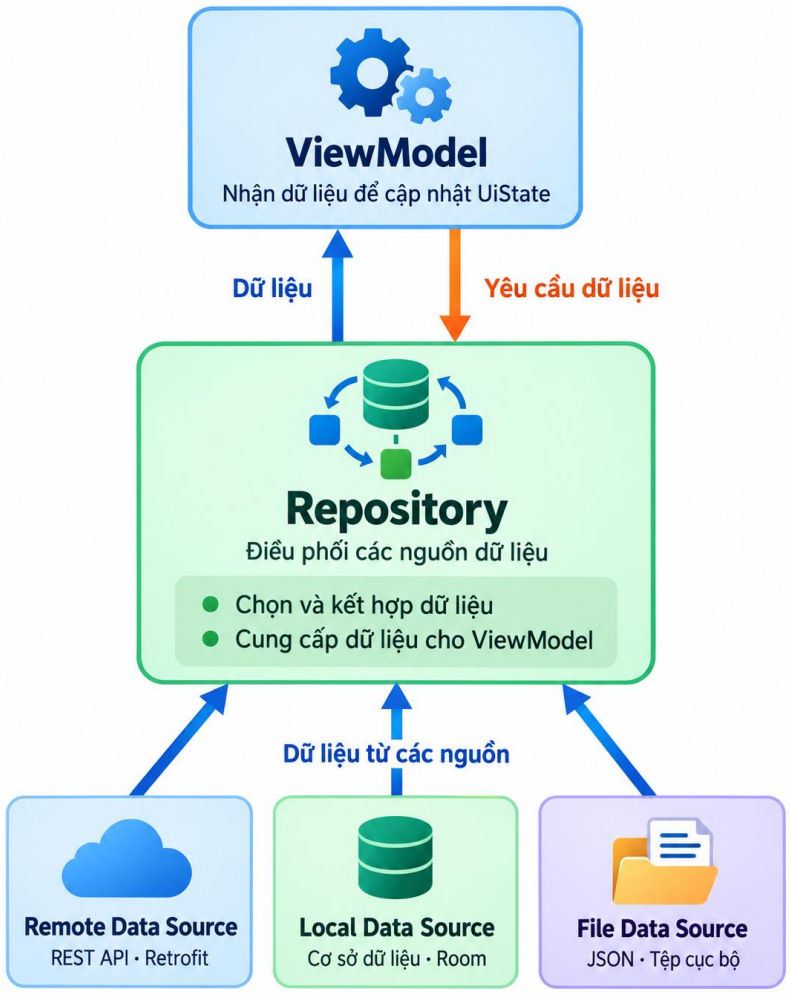
Unidirectional Data Flow

- Dữ liệu chỉ đi theo một chiều, tránh cập nhật trạng thái rải rác.
- UI phát sinh event / action khi người dùng tương tác.
- ViewModel xử lý action, gọi data layer nếu cần, rồi tạo state mới.
- UI chỉ render theo state hiện tại, không tự giữ logic nghiệp vụ phức tạp.
- Lợi ích: dễ debug, dễ test, giảm bug do state không nhất quán.



Repository trong Android Architecture

- Lớp trung gian trong data layer, cung cấp API dữ liệu cho ViewModel.
- Ẩn chi tiết lấy dữ liệu từ REST API, Room database, DataStore hoặc cache.
- Quyết định nguồn dữ liệu phù hợp: đọc cache trước, gọi network khi cần, đồng bộ local với remote.
- Chuyển đổi DTO / entity thành model mà phần còn lại của ứng dụng có thể sử dụng.
- Giúp code dễ test vì ViewModel chỉ phụ thuộc interface repository, không phụ thuộc nguồn dữ liệu cụ thể.



Dependency injection

- Các lớp phụ thuộc vào nhau để có thể hoạt động: **ViewModel** – **Repository** - **ApiService**
- Tình huống: Cần kiểm thử ViewModel mà không thay đổi mã nguồn hiện tại.
- Phương án:
  - Không sử dụng các đối tượng phụ thuộc bên trong lớp => Khai báo ngoài lớp
  - Truyền các đối tượng phụ thuộc qua hàm khởi tạo (constructor injection)

Ví dụ:

```
val repository = NetworkPostRepository(api)
val viewModel = PostViewModel(repository)
```

Tạo lớp repository

```
interface PostRepository {
    suspend fun getPosts(): List<Post>
}


class NetworkPostRepository: PostRepository {
    override suspend fun getPosts(): List<Post> = PostAPI.service.getPosts()
}
```

Việc sử dụng interface cho phép định nghĩa các cách triển khai khác nhau, có thể sử dụng cho mục đích kiểm thử.

Application container

Là đối tượng chứa các phụ thuộc được sử dụng trong ứng dụng: repository, api service, ...

```
class AppContainer {
    val postRepository: PostRepository by lazy {
        NetworkPostRepository()
    }
}
```

Khai báo lớp Container và các đối tượng phụ thuộc bên trong.

```
class PostApplication: Application() {
    lateinit var container: AppContainer
    override fun onCreate() {
        super.onCreate()
        container = AppContainer()
    }
}
```

Khai báo lớp Application

```
<application
    android:name=".PostApplication"
    android:allowBackup="true"
```

Khai báo lớp Application trong file AndroidManifest.xml

Cập nhật ViewModel

```
class PostViewModel(val postRepository: PostRepository): ViewModel() {
    ...

    fun loadPosts() {
        ...
        viewModelScope.launch {
            try {
                val posts = postRepository.getPosts()
                ...
            } catch (e: Exception) {
                ...
            }
        }
    }
}
```

Khai báo Factory để khởi tạo ViewModel

- Lớp ViewModel nhận tham số thuộc tính trong hàm khởi tạo nên không sử dụng cách khởi tạo đối tượng thông thường => Sử dụng **factory** để khởi tạo
- Khai báo **companition object** trong lớp ViewModel

```
companion object {
    val Factory: ViewModelProvider.Factory = viewModelFactory {
        initializer {
            val application = (this[APPLICATION_KEY] as PostApplication)
            val postRepository = application.container.postRepository
            PostViewModel(postRepository)
        }
    }
}
```

Khởi tạo đối tượng ViewModel trong activity

```
val postViewModel: PostViewModel = viewModel(factory = PostViewModel.Factory)
```

Codelab

https://developer.android.com/codelabs/basic-android-kotlin-compose-add-repository

HUST

hust.edu.vn fb.com/dhbkhn

PHẦN 2

- Tìm hiểu về lớp Data trong kiến trúc ứng dụng.
- **Hiển thị ảnh từ internet**

Khai báo thư viện Coil

Thêm vào dependencies trong file build.gradle.kts

```
implementation("io.coil-kt:coil-compose:2.4.0")
```

Hiển thị ảnh sử dụng composable AsyncImage

Ví dụ đơn giản

```
AsyncImage(
    model = "https://lebavui.id.vn/walls/wall1.jpg",
    contentDescription = ""
)
```

Thêm hiệu ứng

```
AsyncImage(
    model = ImageRequest.Builder(LocalContext.current)
        .data("https://lebavui.id.vn/walls/wall1.jpg")
        .placeholder(R.drawable.ic_placeholder)
        .error(R.drawable.ic_error)
        .crossfade(true)
        .build(),
    contentDescription = "",
    contentScale = ContentScale.Crop,
    modifier = Modifier.clip(CircleShape)
)
```

Codelab

https://developer.android.com/codelabs/basic-android-kotlin-compose-load-images

https://developer.android.com/codelabs/basic-android-kotlin-compose-practice-amphibians-app

https://developer.android.com/codelabs/basic-android-kotlin-compose-bookshelf